

Universidad Tecnológica Nacional

Facultad Regional Avellaneda

Departamento de Ingeniería Electrónica

Cátedra: Técnicas Digitales III

Título del Trabajo Práctico:	EGB – Teoría de Modo Protegido
Autores:	Alejandro Eirea Ignacio Mehle

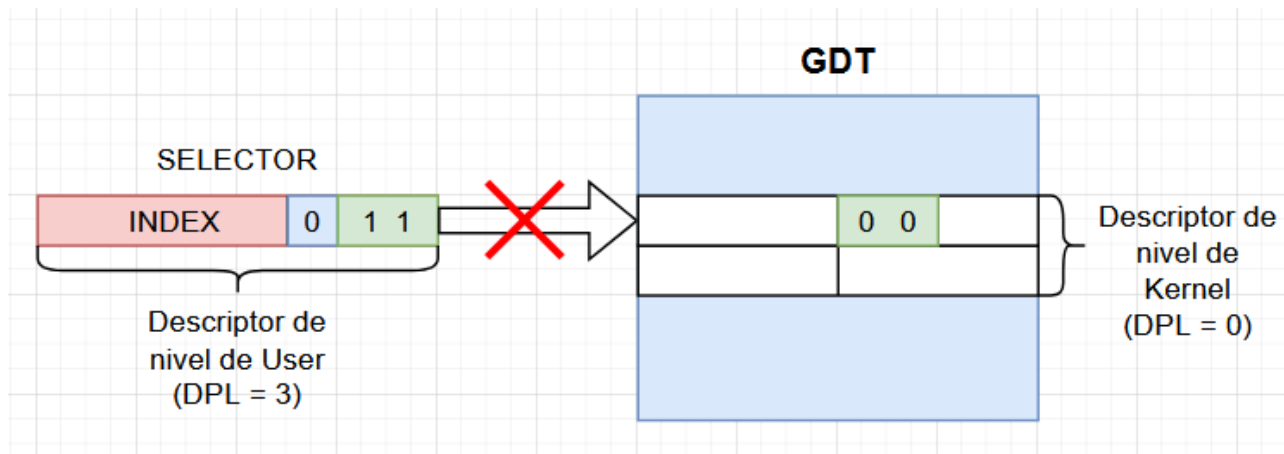
Resumen:

Arquitectura x86: Modo Protegido

1) Mecanismos de Protección de acceso al Segmento en Modo Protegido

- a) Realice un diagrama completo que permita comprender el mecanismo de protección de memoria del 80x386 que permite un no acceso directo al segmento. Explique detalladamente cómo interpreta un valor en un registro de segmento si TI=1 o TI=0

Por ejemplo, si el nivel de privilegio al que se quiere acceder es mayor al del selector, no puede hacerse el acceso y salta una excepción de Protección General.



El bit TI se encuentra en 0 porque se quiere acceder a un descriptor de GDT. El bit TI puesto en 1 indica que el selector apunta a un descriptor de LDT.

- b) ¿Cuáles son las condiciones que deben cumplirse para que el selector correspondiente pueda acceder al segmento? ¿Cuál es la relación entre RPL y DPL?

Para que el selector pueda acceder al segmento debe cumplirse que:

- El selector no debe estar vacío (NULL).
- Su índice debe estar dentro de la GDT o LDT, según corresponda.
- El descriptor debe ser de un tipo válido para ese selector, por ejemplo:
 - Para un selector en DS, el descriptor debe tener en los bits de atributos el bit S=1 (segmento de código o datos), E=0 (no es ejecutable) y ED=0 (segmento de datos).
- El RPL del selector debe ser igual o mayor que el DPL del descriptor de segmento al que se quiere acceder.
- El bit P (Present) debe estar en 1.
- El offset debe estar dentro del límite del segmento.

El Request Privilege Level (RPL) está indicado por los bits 1 y 0 del selector. El nivel de privilegio permite el acceso o no al segmento por parte del selector, indicando un nivel de prioridad que va desde 0 (0b00, más privilegiado), a 3 (0b11, menos privilegiado).

Cada segmento tiene su nivel de prioridad indicado en su descriptor por los bits de DPL (Descriptor Privilege Level). Si el nivel de prioridad del selector tiene igual o mayor nivel de privilegio que el del descriptor, el segmento puede accederse.

Por ejemplo, el kernel utiliza un nivel de privilegio 0, mientras que el espacio de usuario utiliza un nivel de privilegio 3.

LDTR (Local Descriptor Table Register): Es el registro interno de la CPU que guarda el selector de la Tabla Local de Descriptores (LDT) de la tarea que se encuentra en ejecución. Además, se guarda en caché el descriptor de la LDT a la que apunta el GDTR.

Bits de Derechos de Acceso

Bit A: Acceded, lo coloca en 1 el procesador la primera vez que accede al segmento.

Desglosado en binario, el selector contiene:

Index = 0x187F TI = 1 RPL = 11

The diagram illustrates the LDT loading process. It shows the GDT (Global Descriptor Table) and LDT (Local Descriptor Table) structures. The GDT contains a 'DESCRIPTOR DE LDT' entry. The LDT contains an 'OFFSET' and a 'SELECTOR 187F'. The process involves loading the LDT into the GDT and then using the selector to access the LDT content.

2) Descriptores de Segmento y Descriptores de Sistema

- a) ¿Cuántas tablas de descriptores existen en la arquitectura 80x386? Explique cada una de ellas. NOTA: No es lo mismo “tablas de descriptores” que “descriptores”.

En la arquitectura x386 existen 3 tablas de descriptores:

- GDT (Global Descriptor Table): Contiene los descriptores globales, como descriptores de segmentos de código o datos del Sistema Operativo, los punteros a LDT de las tareas, y los TSS. Su address se guarda en el registro GDTR.
 - IDT (Interrupt Descriptor Table): Contiene para las interrupciones y excepciones. Su address se guarda en el registro IDTR.
 - LDT (Local Descriptor Table): Hay una LDT por tarea/proceso, contiene segmentos privados de la tarea, y su address se guarda en el registro LDTR, que contiene un selector a un descriptor de LDT dentro de la GDT.
- b) Explique de qué forma actúa el bit S como atributo de un descriptor. Utilice esta información para decir en qué tipo de descriptor se convierte el mismo. ¿Qué “tipos” de estos descriptores existen?

El bit S de un descriptor hace la diferencia entre descriptores de Sistema, cuando el bit S = 0, y descriptores de Código/Datos, cuando vale 1.

Bit S = 1 (Descriptores de Código/Datos)

Si el bit E (Executable) está en 0, el segmento es de Datos/Pila, y puede clasificarse según:

TIPO (hex)	Bit ED	Bit W	Bit A	Significado
0h	0	0	0	Datos, Sólo Lectura
1h	0	0	1	Datos, Sólo Lectura, accedido
2h	0	1	0	Datos, Lectura/Escritura
3h	0	1	1	Datos, Lectura/Escritura, accedido
4h	1	0	0	Pila, Sólo Lectura
5h	1	0	1	Pila, Sólo Lectura, accedido
6h	1	1	0	Pila, Lectura/Escritura
7h	1	1	1	Pila, Lectura/Escritura, accedido

Si el bit E está en 1, el segmento es de código y se clasifica como:

TIPO (hex)	Bit C	Bit R	Bit A	Significado
8h	0	0	0	Código, Sólo Ejecución
9h	0	0	1	Código, Sólo Ejecución, accedido
Ah	0	1	0	Código, Ejecución/Lectura

TIPO (hex)	Bit C	Bit R	Bit A	Significado
Bh	0	1	1	Código, Ejecución/Lectura, accedido
Ch	1	0	0	Código, Conforme, Sólo Ejecución
Dh	1	0	1	Código, Conforme, Sólo Ejecución, acc.
Eh	1	1	0	Código, Conforme, Ejec/Lectura
Fh	1	1	1	Código, Conforme, Ejec/Lectura, acc.

Bit S = 0 (Descriptores de Sistema)

Tipo (hex)	Nombre	Atributos principales	Uso típico
01h	TSS 16-bit (Disponible)	Segmento de TSS 80286, no ocupado	Mantiene contexto de tarea en sistemas basados en 286.
02h	LDT	Descriptor de LDT	Define tabla local de descriptores para un proceso/tarea. Solo puede estar en GDT.
03h	TSS 16-bit (Ocupado)	Igual a 01h, pero "busy"	Marca que la tarea está activa; evita reentrada.
04h	Call Gate 16-bit	Selector + offset 16-bit + Word Count	Llamada controlada a otro segmento de código, posible cambio de privilegio.
05h	Task Gate	Selector a TSS	Cambio de tarea indirecto, usado en IDT o GDT/LDT.
06h	Interrupt Gate 16-bit	Selector + offset 16-bit	Entrada a ISR. Al activarse, deshabilita interrupciones (IF=0).
07h	Trap Gate 16-bit	Selector + offset 16-bit	Igual a interrupt gate, pero mantiene IF=1 (permite interrupciones anidadas).
09h	TSS 32-bit (Disponible)	Segmento de TSS extendido a 386	Guarda todo el estado extendido de tarea (registros 32-bit, control, etc.).
0Bh	TSS 32-bit (Ocupado)	Igual a 09h pero "busy"	Indica tarea en ejecución.
0Ch	Call Gate 32-bit	Selector + offset 32-bit + Word Count	Igual a 04h pero con punteros 32-bit. Permite saltos controlados entre privilegios.
0Eh	Interrupt Gate 32-bit	Selector + offset 32-bit	Entrada a ISR de 386. IF=0 al entrar.
0Fh	Trap Gate 32-bit	Selector + offset 32-bit	Entrada a manejador que no limpia IF.

Las compuertas o "Gates" se clasifican en:

- Call Gates

Se usan para hacer llamadas a procedimientos (subrutinas) que están en un nivel de privilegio distinto. Permiten que un programa en nivel de usuario (CPL=3) invoque una rutina del sistema operativo en nivel supervisor (CPL=0), sin comprometer la seguridad. El descriptor contiene el selector y el offset del procedimiento destino, además del nivel de privilegio requerido (DPL).

- Task Gates

Se usan para hacer un cambio de tarea (task switch). En lugar de apuntar a código, apuntan a un TSS (Task State Segment). Al activarse, el procesador guarda el contexto actual y carga automáticamente el nuevo contexto de la tarea destino.

Se pueden usar en la GDT, LDT o IDT, por ejemplo, para que una interrupción dispare directamente un cambio de tarea.

- Interrupt Gates

Se definen en la IDT (Interrupt Descriptor Table). Sirven para atender interrupciones de hardware o software. Al activarse, el procesador deshabilita la bandera IF (para evitar interrupciones anidadas) y hace un far jump al manejador indicado.

Son la forma estándar de despachar interrupciones en sistemas protegidos.

- Trap Gates

Muy similares a las Interrupt Gates, pero no limpian la bandera IF al entrar. Se usan para excepciones y trampas de depuración, donde puede ser útil permitir interrupciones anidadas. También están en la IDT.

c) Explique el concepto y funcionamiento de un “Call via Gate”.

Las Call Gate se utilizan cuando se quiere acceder a una subrutina en un nivel de privilegio distinto al actual, por ejemplo a un servicio del Sistema Operativo.

El procesador puede despachar una tarea de las siguientes formas posibles:

- Por medio de una instrucción CALL Por medio de una instrucción JMP
- Mediante una llamada implícita del procesador al handler de una interrupción manejado por una tarea.
- Mediante una llamada implícita del procesador al handler de una excepción manejado por una tarea.
- Mediante la ejecución de la instrucción IRET en una tarea cuando el flag NT (bit 14 del registro EFLAGS) es “1” para la tarea actual.

Se necesita un selector en la GDT que apunte a una puerta de tarea o a un Task State Segment (TSS). Este selector debe estar en la correspondiente posición dentro de la instrucción CALL o JMP.

Antes de iniciar la conmutación, el procesador verifica que $CPL \geq DPL$ del call gate. Para conmutar, se guardan los registros SS, ESP, CS y EIP en stack. Desde el TSS, carga los nuevos selectores de stack (SS, ESP), guarda en el campo Parameter Count los parámetros de la tarea anterior a la nueva pila, y carga el nuevo selector del segmento de código con su EIP de la call gate.

Cuando se conmuta a una tarea mediante un CALL, una interrupción, o una excepción, el procesador copia el TR de la tarea actual en el campo “Previous Task Link” del TSS de la nueva tarea, y luego de completar el cambio de contexto, setea el bit NT del registro EFLAGS (bit 14).

De este modo si la nueva tarea ejecuta en algún punto la instrucción IRET y el bit NT es ‘1’, el procesador conmuta a la tarea anterior ya que tiene el selector de TSS de la tarea previa almacenado en el campo Previous Task Link del TSS de la tarea en ejecución.

En cambio si la conmutación de tarea se efectuó con un JMP no se afecta el flag NT ni se completa el campo “Previous Task Link”.

El procesador utiliza el Bit Busy de un descriptor de TSS para prevenir la reentrancia en una tarea (esto ocasionaría la pérdida de los datos del contexto de ejecución).

Cuando se avanza en anidamiento de tareas mediante un CALL o una interrupción, este bit debe permanecer seteado, en el descriptor de TSS de la tarea previa.

El procesador generará una Excepción de Protección General #GP, si se intenta despachar mediante un CALL o una interrupción una tarea en cuyo TSS está seteado el bit Busy. Esto no

ocurre si la tarea en cuestión se despacha con un IRET ya que es de esperar en esta condición que el bit Busy esté seteado.

Cuando la tarea ejecuta IRET o bien mediante un JMP despacha una nueva tarea, el procesador asume que la tarea actual finalizará y se debe limpiar el bit Busy en su descriptor de TSS.

3) Segmento de Estado de la Tarea (TSS)

- a) ¿Qué se busca con el uso del TSS relacionado al uso del procesador y la conmutación de tareas?

El TSS (Task State Segment) permite guardar y restaurar automáticamente el estado completo de una tarea. El TSS contiene:

- Registros generales
- Registros de segmento
- EIP
- EFLAGS
- Registros de control
- 4 selectores de stack, para cada nivel de privilegio

Esto hace que, cuando el CPU cambia de tarea, el estado de la tarea queda preservado automáticamente por hardware, sin que el SO tenga que cargar registro por registro manualmente.

- b) Explique el funcionamiento y realice un diagrama de interpretación del mismo. Esquema del TSS.

Con un CALL far, JMP far o interrupción a una Task Gate, el procesador guarda el TSS de la tarea actual, carga los registros desde el TSS de la nueva tarea, y salta al EIP de la nueva tarea para continuar la ejecución.

4) Paginado

- a) Explique conceptualmente para qué sirve el proceso de paginado y cómo se relaciona con el sistema de segmentado.

El **paginado** es un mecanismo de administración de memoria que permite mapear direcciones lineales a direcciones físicas de manera flexible. Su principal objetivo es implementar **memoria virtual**, protección de memoria y un uso eficiente de la RAM, evitando la fragmentación externa. Cada página posee atributos de acceso (presente, lectura/escritura, usuario/supervisor), lo que permite al sistema operativo aislar procesos, proteger el kernel y gestionar dinámicamente qué partes de la memoria están realmente en RAM o en almacenamiento secundario.

La **segmentación**, en cambio, organiza la memoria en regiones lógicas (código, datos, pila) y aplica protección y privilegios a nivel de segmento. En procesadores como el 80386, ambas técnicas se encadenan: primero la segmentación traduce un selector y un offset en una **dirección lineal**, y luego el paginado traduce esa dirección lineal en una **dirección física**. En los sistemas operativos modernos la segmentación se usa de forma plana, mientras que el paginado se convierte en el mecanismo central de administración y protección de memoria.

- b) ¿Qué hace la unidad de paginado? ¿Cómo lo hace?

El 80386 utiliza dos niveles de tablas para traducir las direcciones lineales (que vienen de la unidad de segmentación) en una dirección física. Los tres componentes del mecanismo de paginado son: el directorio de páginas, las tablas de páginas y las páginas mismas. Cada uno de

estos elementos ocupa 4KB en la memoria física. Un tamaño uniforme para todos los elementos simplifica el manejo de memoria, ya que no existe fragmentación.

